Homomorphic Encryption Schemes in Finance

Guilherme Marques Alves
Luís Silva

February 8, 2026

1 Introduction

This project focuses on exploring and implementing different Fully Homomorphic Encryption (FHE) schemes in a financial context, with the primary objective of demonstrating secure computation and comparing the efficiency of the chosen schemes.

1.1 Theory Overview

Homomorphic Encryption (HE) is a form of encryption that allows specific computations to be performed directly on ciphertexts, generating an encrypted result that, when decrypted, matches the result of operations performed on the plaintext. This paradigm enables secure computation in untrusted environments (e.g., cloud) without exposing the sensitive data.

The schemes are typically categorized into three types based on their computational capabilities:

- **Partially Homomorphic Encryption (PHE):** Supports only a single type of operation (either addition or multiplication) on encrypted data an unlimited number of times.
- **Somewhat Homomorphic Encryption (SWHE):** Supports both addition and multiplication but is limited to a certain number of operations (depth) before the noise in the ciphertext becomes too large for decryption .
- **Fully Homomorphic Encryption (FHE):** Supports both addition and multiplication for an unlimited number of operations.

In this project, we focus on two specific FHE schemes:

1. **Brakerski-Fan-Vercauteren (BFV):** An exact arithmetic scheme that operates on integers. It provides precise results but requires scaling techniques to handle floating-point numbers.
2. **Cheon-Kim-Kim-Song (CKKS):** An approximate arithmetic scheme designed for real numbers. It treats data as vectors of complex numbers and introduces a small, manageable error in computations, making it highly efficient for statistical and financial analysis.

1.2 Problem and Motivation

An investor (Data Holder) wants to get some statistics about his portfolio (average returns, risk, or performance metrics; in this project, we will only use averages since it depends only on sums and multiplication by a scalar) using an algorithm created by a FinTech (Data Analyzer). The investor does not want to compromise its asset values.

In this project, the specific task chosen is the calculation of the average portfolio returns. This metric requires the fundamental FHE operations of summation and multiplication by a scalar (division by the number of assets). The implementation follows the standard HE workflow:

- Data Holder encrypts data.
- Data Analyzer computes the sum (or product) on the ciphertext.
- Data Holder decrypts the final result to obtain the average.

1.3 Choosing the dataset

We chose the same seed number for D_{100}, D_{1000}, and D_{10000}. This results in:

$$D_1 \subset D_2 \subset D_3 \tag{1}$$

This will be necessary since it will obtain the same data.

2 Implementation

2.1 CKKS

2.1.1 Defining the context

This section initializes the CKKS cryptographic context. We define security parameters such as the polynomial degree, modulus bit sizes, and the global scale necessary for handling real numbers.

```
def setup_ckks_context():
    context = ts.context(
    ts.SCHEME_TYPE.CKKS,
    poly_modulus_degree=8192,
    coeff_mod_bit_sizes=[60, 40, 40, 60]
)
    context.global_scale = 2**40
    context.generate_galois_keys()
    context.generate_relin_keys()
    context.auto_relin = True
    context.auto_rescale = True
    return context
ckks_context = setup_ckks_context()
```

2.1.2 Encryption

The Data Holder encrypts the data. This function reads market values, converts them into a CKKS ciphertext vector, and measures the encryption time for benchmarking. The ciphertext is then serialized and saved.

```
def encrypt_data(context, input_filename, output_filename):
    try:
        df = pd.read_csv(input_filename)
        values = df['Market_Value'].tolist()
    except FileNotFoundError:
        return None
    start_time = time.time()
    try:
        enc_vector = ts.ckks_vector(context, values)
    except Exception as e:
        return None
    with open(output_filename, 'wb') as f:
        f.write(enc_vector.serialize())
    end_time = time.time()
    print(f"[DH] Encrypted data saved as {output_filename}")
    print(f"[DH] Encryption time: {end_time - start_time:.4f} seconds.")
    return enc_vector
```

2.1.3 Data Analysis

The Data Analyzer performs the homomorphic computation. It loads the public context and the encrypted vector, performs the summation, and includes a multiplication by 1.0 for benchmarking purposes. The analysis time is recorded.

```
PUBLIC_CONTEXT_FILE = "public_context.bin"
def analyze_ckks_benchmark(enc_data_filename, res_data_filename):
    try:
        with open(PUBLIC_CONTEXT_FILE, 'rb') as f:
            context = ts.Context.load(f.read())
    except FileNotFoundError:
        return None
    try:
        with open(enc_data_filename, 'rb') as f:
            enc_vector = ts.ckks_vector_from(context, f.read())
    except FileNotFoundError:
        return None
    start_time = time.time()
    encrypted_sum = enc_vector.sum()
    factor_benchmark = 1.0
```

```
    encrypted_result = encrypted_sum * factor_benchmark

    with open(res_data_filename, 'wb') as f:
        f.write(encrypted_result.serialize())
    end_time = time.time()
    print(f"[DA-CKKS] Analysis (Sum + Mult 1.0) concluded in: {end_time - start_time:.4f} seconds.")
    return end_time - start_time
INPUT_ENC_FILE = "encrypted_data.bin"
OUTPUT_RES_FILE = "encrypted_results.bin"
time_analysis_ckks = analyze_ckks_benchmark(INPUT_ENC_FILE, OUTPUT_RES_FILE)
```

2.1.4 Decryption and Comparison

The Data Holder decrypts the final result. This function retrieves the encrypted sum, performs the decryption, calculates the final average by dividing the number of assets, and compares it against the real average for precision validation.

```
def decrypt_results_ckks(context, res_data_filename, n_assets):
    if not context.has_secret_key():
        raise ValueError("ERRO: No secret key.")
    try:
        with open(res_data_filename, 'rb') as f:
            encrypted_result_bytes = f.read()
    except FileNotFoundError:
        return None
    start_time = time.time()
    enc_result_vector = ts.ckks_vector_from(context, encrypted_result_bytes)
    sum_decrypted = enc_result_vector.decrypt(secret_key=context.secret_key())[0]
    mean_final = sum_decrypted / n_assets

    end_time = time.time()

    print(f"[Client-CKKS] Decryption and Division completed in {end_time - start_time:.4f} seconds.")
    return mean_final

N_ASSETS = 5000
OUTPUT_RES_FILE = "encrypted_results.bin"

try:
    final_value_ckks = decrypt_results_ckks(ckks_context, OUTPUT_RES_FILE, N_ASSETS)

    print(f"CKKS Average: {final_value_ckks:.6f}")
    df = pd.read_csv("portfolio_data5000.csv")
    real = df['Market_Value'].mean()
    print(f"Real Average:  {real:.6f}")
    print(f"Diff:  {abs(real - final_value_ckks):.6f}")

except Exception as e:
    print(f"Error: {e}")
```

2.2 Brakerski Fan-Vercauteren (BFV)

2.2.1 Context

We now set up the context for the BFV scheme (exact integer arithmetic). This requires defining the plaintext modulus and generating the necessary keys for secure operations.

```
    def setup_bfv_context():
    context = ts.context(
        ts.SCHEME_TYPE.BFV,
        poly_modulus_degree=8192,
        plain_modulus=1032192001
    )
    context.generate_galois_keys()
```

```
    context.generate_relin_keys()
    return context

bfv_context = setup_bfv_context()
```

2.2.2 Encryption

BFV encryption requires converting floats to integers by applying a scale factor. Due to BFV limitations, the data must be divided into chunks, and each chunk is encrypted separately. This introduces the overhead measured in our analysis.

```
SCALE_FACTOR = 10
CHUNK_SIZE = 4000

def encrypt_data_bfv_chunked(context, input_filename, output_filename, scale):
    try:
        df = pd.read_csv(input_filename)
        values = df['Market_Value'].tolist()
    except FileNotFoundError:
        print("Erro: Ficheiro no encontrado.")
        return None
    start_time = time.time()

    scaled_values = [int(round(v * scale)) for v in values]


    chunks = [scaled_values[i:i + CHUNK_SIZE] for i in range(0, len(scaled_values),
        CHUNK_SIZE)]


    encrypted_chunks = []
    for chunk in chunks:

        enc_vec = ts.bfv_vector(context, chunk)
        encrypted_chunks.append(enc_vec.serialize())
    with open(output_filename, 'wb') as f:
        pickle.dump(encrypted_chunks, f)
    with open("public_context_bfv.bin", 'wb') as f:
        f.write(context.serialize(save_secret_key=False))

    end_time = time.time()
    print(f"[DH-BFV] Encryption done in {end_time - start_time:.4f} seconds.")
    return end_time - start_time


OUTPUT_ENC_BFV = "encrypted_data_bfv.pkl"
time_enc_bfv = encrypt_data_bfv_chunked(bfv_context, "portfolio_data5000.csv",
    OUTPUT_ENC_BFV, SCALE_FACTOR)
```

2.2.3 Analysis

The BFV analysis requires iterating over all encrypted chunks. The function sequentially sums the partial results from each chunk and includes a final multiplication by the identity vector [1] before serializing the total encrypted sum.

```
def analyze_bfv_chunked(enc_filename, res_filename):
    with open("public_context_bfv.bin", 'rb') as f:
        context = ts.Context.load(f.read())
    with open(enc_filename, 'rb') as f:
        encrypted_chunks_bytes = pickle.load(f)

    start_time = time.time()

    total_encrypted_sum = None

    for i, chunk_bytes in enumerate(encrypted_chunks_bytes):
        try:
```

```
            vec = ts.bfv_vector_from(context, chunk_bytes)
        except:
             vec = ts.load(chunk_bytes)
             vec.link_context(context)

        chunk_sum = vec.sum()

        if total_encrypted_sum is None:
            total_encrypted_sum = chunk_sum
        else:
            total_encrypted_sum += chunk_sum


    plain_identity = [1]


    encrypted_result = total_encrypted_sum.mul(plain_identity)
    with open(res_filename, 'wb') as f:
        f.write(encrypted_result.serialize())

    end_time = time.time()
    print(f"[DA-BFV] Analysis (Sum + Mult 1.0) done in {end_time - start_time:.4f} seconds.")
    return end_time - start_time

OUTPUT_RES_BFV = "encrypted_results_bfv.bin"
time_analyze_bfv = analyze_bfv_chunked(OUTPUT_ENC_BFV, OUTPUT_RES_BFV)
```

2.2.4 Decryption and comparison

The BFV decryption process concludes the workflow. After decryption, the final integer sum must be de-scaled (divided by the initial scale factor and the number of assets) to obtain the true real average, which is then validated.

```
def decrypt_bfv_final(context, res_filename, scale, n_assets):
    print(f"\n[Client-BFV] A decifrar...")

    with open(res_filename, 'rb') as f:
        try:
            enc_result = ts.bfv_vector_from(context, f.read())
        except:
            enc_result = ts.load(f.read())
            enc_result.link_context(context)

    start_time = time.time()


    sum_int = enc_result.decrypt()[0]


    real_average = (sum_int / n_assets) / scale

    end_time = time.time()
    print(f"[Client-BFV] Decryption completed in {end_time - start_time:.4f}s.")
    return real_average

N_ASSETS = 5000
SCALE_FACTOR = 10

final_mean_bfv = decrypt_bfv_final(bfv_context, OUTPUT_RES_BFV, SCALE_FACTOR, N_ASSETS)

print(f"BFV Average: {final_mean_bfv:.4f}")


try:
    df_check = pd.read_csv("portfolio_data5000.csv")
```

```
    real_mean = df_check['Market_Value'].mean()
    print(f"Real mean:      {real_mean:.4f}")
    print(f"difference:       {abs(real_mean - final_mean_bfv):.6f}")
except:
    print("Original file not found for verification.")
```

3 Results

3.1 Cheon-Kim-Kim-Song (CKKS)

Dataset Size (N)	Encryption (s)	Analysis (s)	Decryption (s)	Difference (%)	Total Time (s)
100	0.0074	0.0159	0.0025	0.000013	0.0258
1000	0.0075	0.0401	0.0017	0.000013	0.0493
5000	0.0175	0.0309	0.0015	0.000013	0.0499
10000	0.0185	0.0387	0.0022	0.000013	0.0594

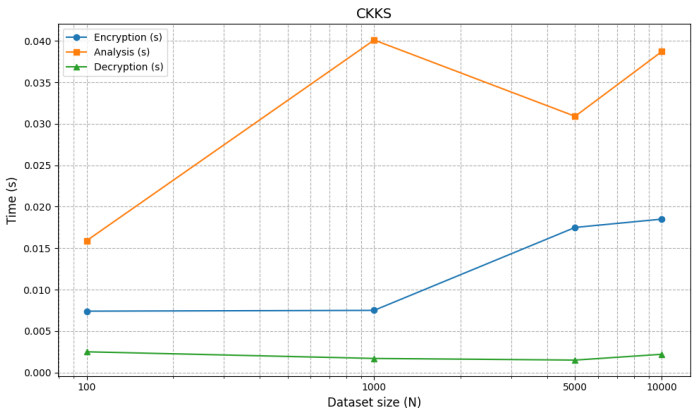
Table 1: Performance metrics for the CKKS scheme across different dataset sizes.

3.2 Brakerski Fan-Vercauteren (BFV)

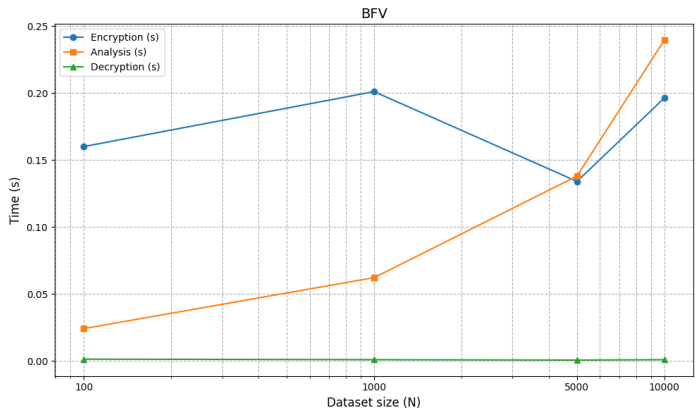
Dataset Size (N)	Encryption (s)	Analysis (s)	Decryption (s)	Difference (%)	Total Time (s)
100	0.1600	0.0242	0.0014	0	0.1856
1000	0.2010	0.0622	0.0010	0	0.2642
5000	0.1340	0.1380	0.0007	0	0.2727
10000	0.1963	0.2395	0.0010	0	0.4368

Table 2: Performance metrics for the BFV scheme across different dataset sizes.

3.3 Benchmarking



(a) CKKS: Disaggregated execution times.



(b) BFV: Disaggregated execution times.

3.4 Efficiency

Based on the results provided by the tables:

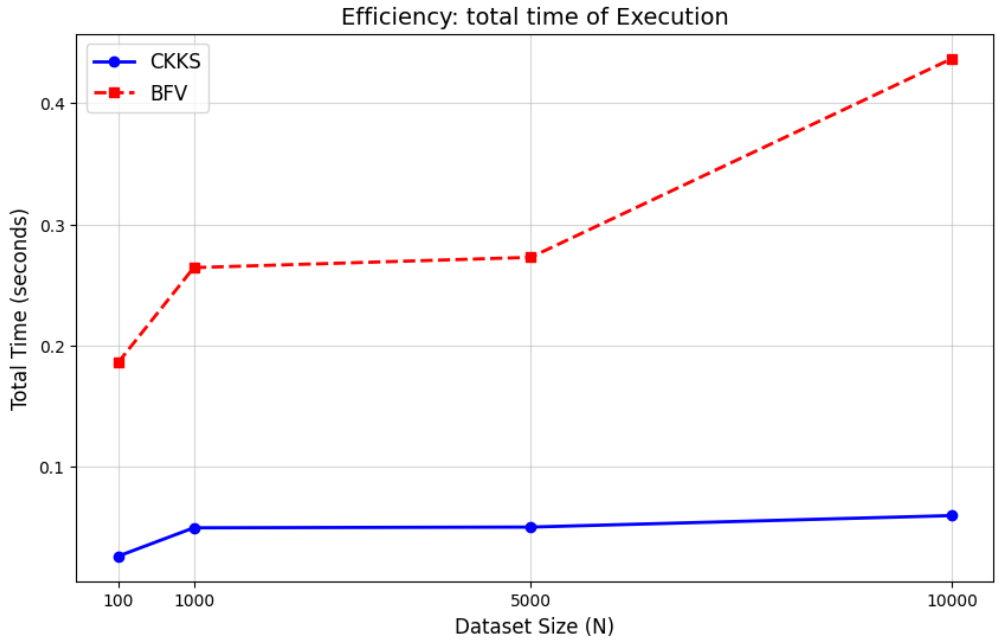


Figure 2: Efficiency based on the total time of execution

4 Discussion

Following the results based on figure, we can see that CKKS is sistematically faster than BFV, which was expected since the operations were simpler and provided an approximation.

There is also the limitations of using BFV, in which we can not do calculations using floats, for benchmark reasons we used a scale factor which scaled our floats into ints. This scale factor, if big enough can change the whole result (we tested for scale factor = 100 and the difference of 0.0002 turned out to almost 500 in the final result). Also most of the statistical analysis tools and metrics needs multiplication or division by non-integers which makes it almost impossible to use in financial contexts.

The systematic efficiency gap, clearly visible in Figure 2, is primarily driven by the Encryption and Analysis phases. BFV demonstrated a significantly higher encryption overhead, taking approximately 26 times longer than CKKS for the $N = 1000$ dataset. This latency is due to the BFV scheme on outputing exact values. To handle our floating-point portfolio values, BFV required the manual scaling of values and, critically, the division of the dataset into multiple ciphertext blocks (chunks) (e.g., using a CHUNK_SIZE of 4000). This iteration over multiple chunks during the encryption process significantly increases the Data Holder's computational power needed.

In contrast, CKKS is an approximate arithmetic scheme designed for real numbers, which allows it to naturally handle our financial data and utilize efficient slot packing to encode a large vector of values into a single ciphertext, resulting in minimal encryption time. This representation over vectors of complex numbers is fundamental to its ability to handle floating-point arithmetic. Conversely, the BFV scheme's strict requirement to operate over polynomial rings ensures arithmetic exactness but necessitates the scaling and chunking workflow for non-integer data.

Furthermore, the time for the Analysis operation (Sum + Multiplication by 1) in BFV also shows a steeper increase, especially noticeable when moving from $N = 5000$ to $N = 10000$. This is because the Data Analyzer is forced to sequentially process and aggregate the results from each encrypted chunk, whereas CKKS processes the single encrypted vector more efficiently. This overhead explains why the BFV Analysis time jumps significantly between $N = 5000$ (0.1380s) and $N = 10000$ (0.2395s), as the number of sequential chunks/batches that need to be processed increases proportionally to the dataset size. The inclusion of multiplication by a factor of 1.0 (or [1] in BFV) was necessary to fulfill the project requirement of including both operations and allowed for an accurate benchmark, which is crucial for assessing real-world performance.

Despite the superior efficiency of CKKS, its results are approximate. However, the recorded error (ϵ) is consistently small, in the order of the fourth decimal place (e.g., 0.000203 for $N = 5000$), a level of precision that is generally acceptable and manageable for statistical and financial reporting.

5 Conclusion

We conclude that the CKKS scheme is the definitive choice for this project's financial analysis context. While BFV guarantees exact results, its implementation introduced significant practical limitations: it was up to 26 times slower in encryption for $N = 1000$ due to the overhead of scaling non-integer values and processing data in multiple sequential chunks. Furthermore, the need for chunking and managing the scale factor complicates the workflow, potentially compromising security and privacy by requiring communication beyond the encrypted computation.

In contrast, CKKS offers a highly efficient solution, providing excellent approximations (with an error consistently in the order of the fourth decimal place) and being natively suited for the real numbers used in financial statistics. Thus, the substantial gain in efficiency and usability far outweighs the negligible loss of precision.